



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

FPGA ACCELERATOR FOR LLM

REPORT OF WEEK 7

Work of :

GAUVRIT Jean-Luc

As intern student

With the supervision of :

KIM Tae-Wan

JULY 2025

Contents

1	Introduction	2
1.1	Presentation of the Weekly Report	2
1.2	Presentation of the Subject	2
1.3	The model	2
1.4	FPGA	3
2	LLaMa	4
2.1	What is LLaMA?	4
2.2	Inference of LLaMA	4
3	Inference on CPU	6
3.1	Inference C++	6
3.2	Main and Build System	7
3.3	Build, Run & Test	7
3.3.1	CMake-based Build System	7
3.3.2	Command-line Inference	7
3.3.3	Test	8
4	Inference FPGA	9
4.1	PetaLinux Setup	9
4.2	HLS Kernel Compilation and FPGA Bitstream Generation	9
4.3	Board Programming and Deployment	10
5	Docker-Based Inference Automation	11
5.1	Container Structure	11
5.2	Scripted Workflow	11
5.3	Future Extension	12
6	Conclusion	13
7	Annex	14
7.1	Tensors formats	14
7.2	From C++ to HLS	15
	Bibliography	16

List of Figures

1	Logo de DeepSeek	2
2	Transformer of LLaMA	4
3	Table of command-line options for CPU inference	7
4	PetaLinux workflow for deployment and debug on Zynq MPSoC platforms	9
5	FPGA-Based Solution for On-Board Verification of Hardware Modules Using HLS	11
6	Automated compilation and deployment flow using a Docker container.	12
7	Our Milestone	13
8	GGUF [1] binary file layout	14
9	Safetensors architecture	14

1 Introduction

1.1 Presentation of the Weekly Report

This weekly report presents the progress of my work on designing and optimizing a hardware accelerator for Large Language Models (LLMs) using Field-Programmable Gate Arrays (FPGAs). Its objective is to document technical advancements, highlight open design questions, and ensure alignment with the research direction defined in collaboration with Professor Tae-Wan Kim. Each section outlines key investigations, experimental results, and technical challenges encountered during the week.

1.2 Presentation of the Subject

The goal of this project is to do inference with a LLM, specifically a variant of the DeepSeek-R1 family, on a FPGA platform. DeepSeek has recently demonstrated excellent inference performance across various benchmarks, offering competitive accuracy while being more lightweight and efficient than many other open-source LLMs of similar size. These characteristics make it a strong candidate for hardware-level optimization.

LLMs typically require considerable computational power and memory bandwidth, which limits their deployment in edge or low-power environments. While GPUs provide strong performance, their energy consumption is often prohibitive for embedded or mobile use cases.

This project explores the use of FPGAs to design a custom, energy-efficient accelerator optimized for DeepSeek inference. The flexibility of FPGAs allows for fine-tuned architectural choices targeting both compute and memory efficiency. Our focus is on reducing power consumption while maintaining acceptable latency (e.g., 100–200 ms per token), with attention given to constraints such as voltage levels, memory interfaces, and model size.

Our final goal is to develop an Application-Specific Integrated Circuit (ASIC) that is energy-efficient and capable of accelerating LLMs in low-power environments. The development of a Proof of Concept (PoC) using FPGAs is a crucial step towards achieving this goal. The PoC will demonstrate the feasibility and potential benefits of using FPGAs for LLM acceleration, paving the way for future ASIC development.

1.3 The model

The goal of this project is to perform inference with a Large Language Model, specifically a variant of the DeepSeek-R1 family, on an FPGA platform. DeepSeek has recently demonstrated excellent inference performance across various benchmarks, offering competitive accuracy while being more lightweight and efficient than many other open-source LLMs of similar size. These characteristics make it a strong candidate for hardware-level optimization.



Figure 1: Logo de DeepSeek

Ultimately, the objective is to evaluate whether FPGA-based acceleration can provide a sustainable and scalable solution for running LLMs like DeepSeek in compact or embedded systems.

1.4 FPGA

Field-Programmable Gate Arrays (FPGAs) are reconfigurable integrated circuits that can be programmed to implement custom digital logic. Their main advantage lies in flexibility : unlike fixed-function hardware, FPGAs allow designers to create architectures tailored to specific applications, such as neural network inference. This adaptability makes them well suited for achieving an efficient balance between performance, power consumption, and resource usage.

2 LLaMa

2.1 What is LLaMA?

LLaMA, which stands for Large Language Model Meta AI, is a family of transformer-based language models developed by Meta AI. Designed to be open-weight and relatively lightweight compared to proprietary models like GPT-3, LLaMA has become a popular base architecture in the open-source AI community.

The original LLaMA paper [3] introduced a decoder-only transformer model, optimized for efficiency and portability. It supports autoregressive text generation, like GPT-family models, but is designed to be fine-tuned on modest hardware. Since its release, several successors have emerged : LLaMA 2 and LLaMA 3, improving model alignment, scalability, and training dataset quality.

2.2 Inference of LLaMA

One of the main reasons I chose to focus on the LLaMA architecture is that several advanced language models, including **DeepSeek**, are directly based on it. By developing an efficient and functional inference kernel compatible with LLaMA, I gain immediate interoperability with a wide range of state-of-the-art models without needing to reinvent the core computational pipeline.

The LLaMA architecture relies on a well-structured sequence of operations that are common to modern transformer-based models. These include the embedding lookup phase, followed by multi-head attention mechanisms that incorporate rotary positional encodings (RoPE), then feed-forward neural network layers using non-linear activations, normalization steps such as LayerNorm, and finally token sampling and decoding.

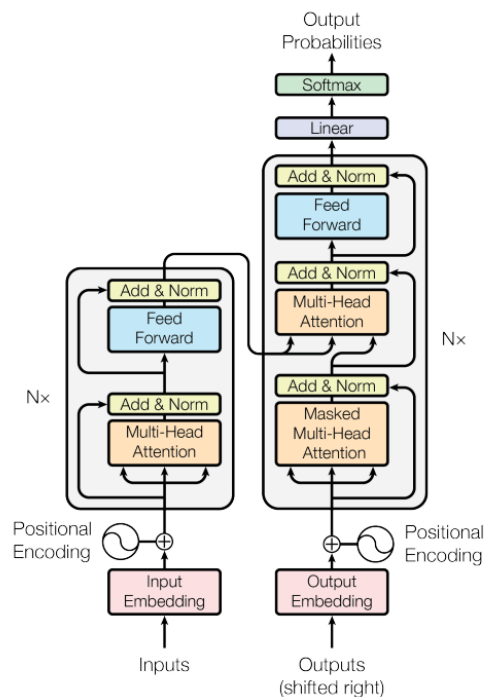


Figure 2: Transformer of LLaMA

This standardized computational graph serves as the backbone for numerous LLaMA-derived models. As a result, an optimized implementation targeting LLaMA can naturally extend to variants such as DeepSeek, Mistral, TinyLLaMA, and others, with minimal architectural modification. This makes LLaMA not only a technical reference but also a strategic entry point for scalable and reusable inference solutions.

For projects such as `llama.cpp` or **Swan**, this standardization is a powerful enabler:

- It allows reusing the same inference backend across various models.
- It guides hardware acceleration (e.g., FPGA) efforts toward a clearly defined and reusable compute structure.
- It ensures compatibility with the ecosystem of quantized models available in the GGUF format.

Targeting LLaMA for inference is thus a highly strategic choice, particularly in the context of building lightweight, hardware-friendly AI pipelines.

3 Inference on CPU

3.1 Inference C++

Building upon the principles of C++ inference as seen in `llama.cpp`, the core engine of the Swan project [2] is also implemented in modern C++. This design choice ensures portability across platforms, high efficiency, and crucially, the ability to work with High-Level Synthesis (HLS) tools for the deployment of FPGAs. The same inference logic can thus be compiled for CPU execution or synthesized into hardware logic for acceleration on FPGAs, depending on the selected backend.

My C++ inference flow is structured as follows :

1. **Loading Weights and Tokenizer:** Binary weight files (e.g., `stories15M.bin`) and the vocabulary file (`tokenizer.bin`) are loaded into memory. These files are downloaded from open repositories and provide the quantized model parameters and the tokenization scheme used during training.
2. **Tokenization:** The input prompt is tokenized using a custom vocabulary implementation (`vocab.cpp/.hpp`) that maps strings to token IDs and vice versa. This lightweight tokenizer is optimized for embedded inference.
3. **Forward Pass:** The tokenized input is processed one token at a time. For each input token, the model performs a forward pass, which includes:
 - Embedding lookup
 - Matrix multiplications (attention and MLP blocks)
 - Activation functions (e.g., ReLU, GELU)
 - Softmax normalization

Depending on the backend, these operations can be computed on the CPU (`tensor.cpp`) or accelerated using FPGA modules (`tensor_fpga.cpp`).

4. **Sampling and Generation:** The output logits are post-processed with temperature scaling and passed through a sampling function to determine the next token. This token is appended to the input sequence, and the process repeats until a maximum length or end-of-sequence token is reached.
5. **Decoding:** The generated sequence of tokens is decoded back into human-readable text using the vocabulary.

The modular structure of the C++ code enables easy substitution of specific operations (e.g., replacing CPU matrix multiplication with an HLS-optimized FPGA kernel), making it ideal for exploring the trade-offs between performance, power, and flexibility in edge AI inference.

This design enables running quantized LLMs in constrained environments while maintaining reasonable accuracy and responsiveness.

3.2 Main and Build System

The file `main.cpp` is the entry point of the inference framework. It coordinates the execution pipeline by:

- Parsing command-line arguments (e.g., weight path, prompt, sequence length).
- Loading model weights and the tokenizer from binary files.
- Tokenizing the input prompt and feeding it into the inference engine.
- Executing the forward pass loop to generate output tokens.
- Decoding the results back into human-readable text.

This design enables a clean separation between infrastructure code and inference logic, facilitating backend switching (CPU or FPGA).

3.3 Build, Run & Test

3.3.1 CMake-based Build System

The project uses a minimalist CMake build configuration. The root `CMakeLists.txt` supports both CPU-only and FPGA-accelerated builds.

To build the project for CPU-only execution (no FPGA), simply run :

```
1 mkdir build && cd build
2 cmake ..
3 make
```

3.3.2 Command-line Inference

Once compiled, inference can be launched via terminal with a prompt and output options :

```
1 ./inference_fpga --prompt "Once upon a time" --max_seq 64 --temp 0.7
```

Available flags include :

Flag	Description	Default
<code>--weight_path</code>	Path to <code>.bin</code> weight file	<code>model/stories15M.bin</code>
<code>--vocab_path</code>	Path to tokenizer binary	<code>model/tokenizer.bin</code>
<code>--prompt, -p</code>	Initial prompt (UTF-8 string)	empty (generate from BOS)
<code>--max_seq</code>	Maximum number of generated tokens (prompt is added on top)	256
<code>--temp</code>	Sampling temperature (≤ 0 means greedy decoding)	0.5
<code>--color</code>	Enable ANSI-colored token output	off
<code>--log</code>	Dump intermediate tensors to <code>./log/</code>	off
<code>--help, -h</code>	Show help and exit	—

Figure 3: Table of command-line options for CPU inference

This CPU fallback mode is ideal for debugging, testing, and profiling without requiring access to FPGA hardware. It also facilitates CI pipelines and quick iterations on the model logic.

3.3.3 Test

To validate the functionality and performance of the C++ inference pipeline, a test was executed on a standard development machine using the CPU-only build. The inference binary was launched with a temperature of 1 and a generation limit of 100 tokens :

```
1 ./build/inference_fpga --temp 1 --max_seq 100
```

Upon execution, the model reports its hyperparameters, which confirm that a 15M-parameter TinyLLaMA model is being used. Key parameters include a hidden dimension of 288, 6 transformer layers, and a vocabulary size of 32,000 tokens. The system generated a coherent text sequence starting with "Once upon a time...", demonstrating proper functioning of the token generation loop.

```
1 Hyper Parameters
2   dim          : 288
3   ffn_dim      : 768
4   n_layers     : 6
5   n_heads      : 6
6   n_kv_heads   : 6
7   vocab_size   : 32000
8   seq_len      : 256
9   Once upon a time, there was a little girl named Lily...
```

The full inference took approximately 4.25 seconds, achieving an average generation speed of **23.5 tokens per second**. This confirms that the CPU-only pipeline is functional and sufficiently fast for development and debugging purposes.

```
1 Time : 4.25001[s]
2 Speed: 23.5293[tok/s]
```

4 Inference FPGA

4.1 PetaLinux Setup

To simplify the integration and execution of inference workloads on the ZCU106 FPGA board, I utilized the **PetaLinux** development tools provided by AMD Xilinx. PetaLinux enables the creation of a fully customized embedded Linux distribution tailored for the Zynq UltraScale+ MPSoC architecture, offering a seamless environment for deploying and running software alongside FPGA hardware acceleration.

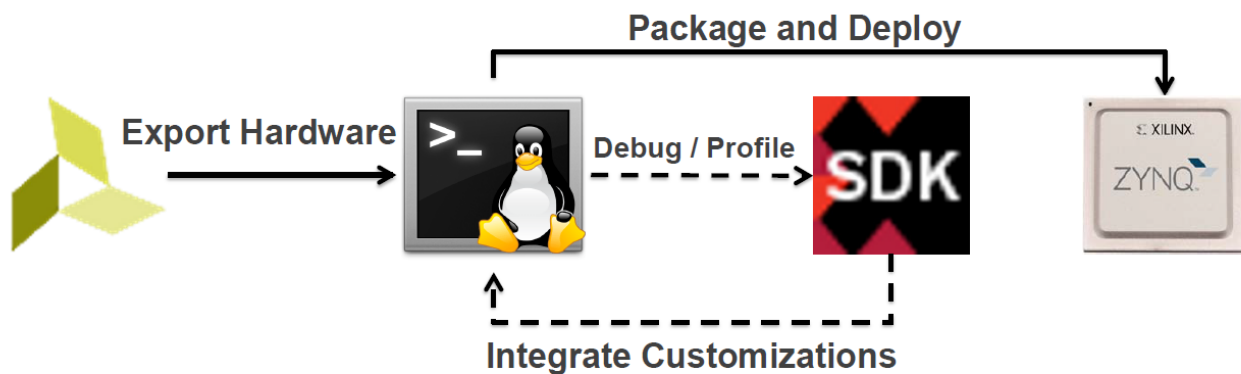


Figure 4: PetaLinux workflow for deployment and debug on Zynq MPSoC platforms

This setup significantly simplifies hardware-software integration and provides a user-space Linux environment to manage input/output operations and load FPGA bitstreams and kernel binaries (.xclbin) for inference.

4.2 HLS Kernel Compilation and FPGA Bitstream Generation

To accelerate the LLM inference on the FPGA, we implemented custom kernels in C++ using High-Level Synthesis (HLS). These kernels include matrix multiplication, RMS normalization, softmax, and other operations.

1. Each kernel (cf figure 2) was written as a standalone C++ file and synthesized using the Vitis HLS toolchain.
2. The following command was used for kernel compilation:

```
1 v++ -c -t hw --platform zcu106_dpu -k kernel_<name> kernel_<name>
  >.cpp -o kernel_<name>.xo
```

3. The individual compiled object files (.xo) were then linked to produce a unified bitstream:

```
1 v++ -l -t hw --platform zcu106_dpu kernel_*.xo -o
  binary_container_1.xclbin
```

4. The final bitstream was extracted from the XCLBIN file and programmed onto the FPGA using Vivado or 'xsct'.

4.3 Board Programming and Deployment

To deploy the system on the ZCU106 board, the generated `BOOT.BIN` file, Linux kernel image (`Image`), and device tree blob (`system.dtb`) were copied to a bootable microSD card. The board was configured to boot from the SD card using its hardware dip-switch settings. Upon powering the board, the system successfully booted into the embedded Linux environment, and UART communication was used to monitor the console output in real time.

Once the operating system was up and running, users could either access the board through a UART terminal or establish an SSH connection, depending on the network configuration. The inference host application, previously cross-compiled using the ARM toolchain, was executed within this Linux environment. This application interacts directly with the FPGA using the XRT runtime and the corresponding `.xclbin` file containing the hardware-accelerated kernels.

This setup enabled us to perform end-to-end inference on the FPGA. We evaluated the performance by measuring runtime metrics and token generation speed across a variety of prompts. The complete system flow—from boot to inference—demonstrated seamless integration between software and hardware, highlighting the effectiveness of the PetaLinux deployment model.

5 Docker-Based Inference Automation

To streamline the deployment process and make the inference pipeline reproducible and portable, I designed a Docker-based automation setup.

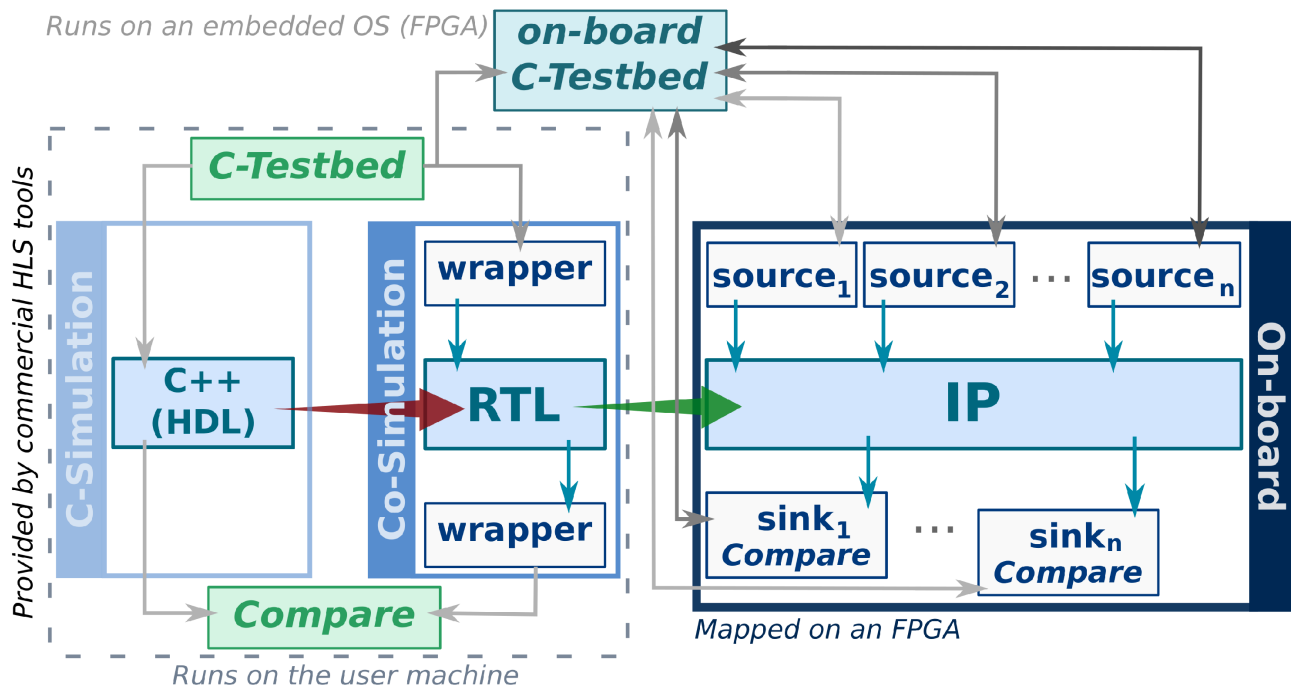


Figure 5: FPGA-Based Solution for On-Board Verification of Hardware Modules Using HLS

This approach allows us to encapsulate the entire compilation and execution workflow inside a container, removing the need for repeated environment configuration or manual intervention on the host machine.

5.1 Container Structure

The Docker container is defined via a custom `Dockerfile` that performs the following steps :

- Installs system dependencies for cross-compilation and Vitis tools.
- Copies the project files including source code, models, and scripts.
- Sources Vitis environment variables via `settings64.sh`.
- Executes a startup script `docker-entrypoint.sh` to compile both hardware (FPGA kernels with V++ compiler cf 7.2) and software (host inference application cross-compiled for ARM).

The container uses the official Ubuntu 22.04 base image and assumes access to Xilinx toolchains (e.g., installed at `/tools/Xilinx/2025.1` inside the container).

5.2 Scripted Workflow

Two key scripts orchestrate the build :

- `compil_hls.sh`: Compiles FPGA kernels using `v++`, then links them into a `.xclbin` binary.
- `run_llm.tcl`: Used with `xsct` to program the board via JTAG, load the `bitstream`, and boot the inference pipeline directly if needed.

All paths and environment variables are automatically handled inside the container. This ensures that inference can be recompiled and deployed on any compatible system with Docker and Xilinx tools installed.

5.3 Future Extension

The current Docker setup is functional for compiling and building inference workloads. In the future, it can be extended to :

- Automate SD card image creation using PetaLinux.
- Launch interactive UART or SSH sessions into the board.
- Monitor inference performance metrics and logs.

This containerized infrastructure is essential for reproducibility, especially in hardware-software co-design environments where setup and dependencies can be difficult to replicate manually.

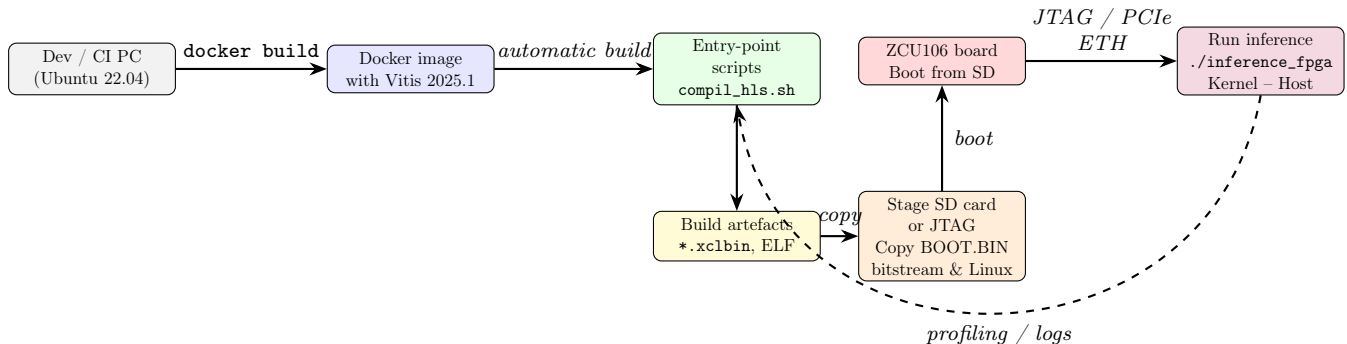


Figure 6: Automated compilation and deployment flow using a Docker container.

6 Conclusion

This report has detailed the construction of a lightweight inference pipeline for LLaMA-based language models, tailored for embedded systems and hardware acceleration on FPGAs. Starting from a C++ kernel inspired by `llama.cpp`, we developed a modular architecture capable of running both in software (on CPU) and in hardware (via High-Level Synthesis).

A significant contribution of this work is the design and implementation of a numerically stable and hardware-friendly `softmax` function. Instead of relying on costly exponential and division operations, our version uses fixed-point polynomial approximations, making it compatible with synthesis for low-resource devices. This logic was successfully converted to VHDL and verified using a finite-state machine (FSM).

We also developed a complete inference stack in modern C++, including prompt tokenization, quantized weight loading, and autoregressive token generation. This implementation ensures correctness and provides a robust reference for validating hardware results.

At this point, we are **very close to running inference entirely on FPGA**. The core kernels have been compiled, linked into a bitstream, and integrated with a host application. The remaining work involves stress-testing the pipeline under real input conditions and validating stability and throughput.

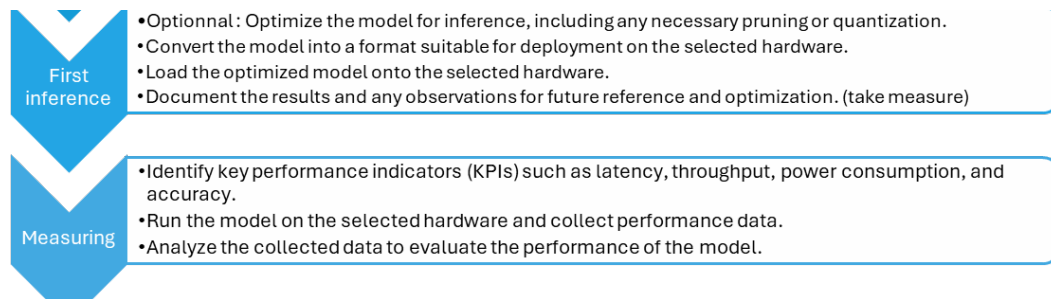


Figure 7: Our Milestone

Although we are slightly behind the projected milestone plan, the current progress is promising. A benchmarking phase is still required to evaluate:

- The performance of the new C++ inference backend,
- The comparative efficiency of the softmax implementations (standard, Taylor-approximation, log-domain).

This benchmarking step is essential to guide final design choices for deployment and will provide quantitative insights into latency and resource usage. Once complete, we will be able to run full inference directly on the FPGA platform.

In summary, the foundation is solid, the architecture is in place, and only a few final steps remain to complete an end-to-end FPGA-based inference engine for large language models.

7 Annex

7.1 Tensors formats

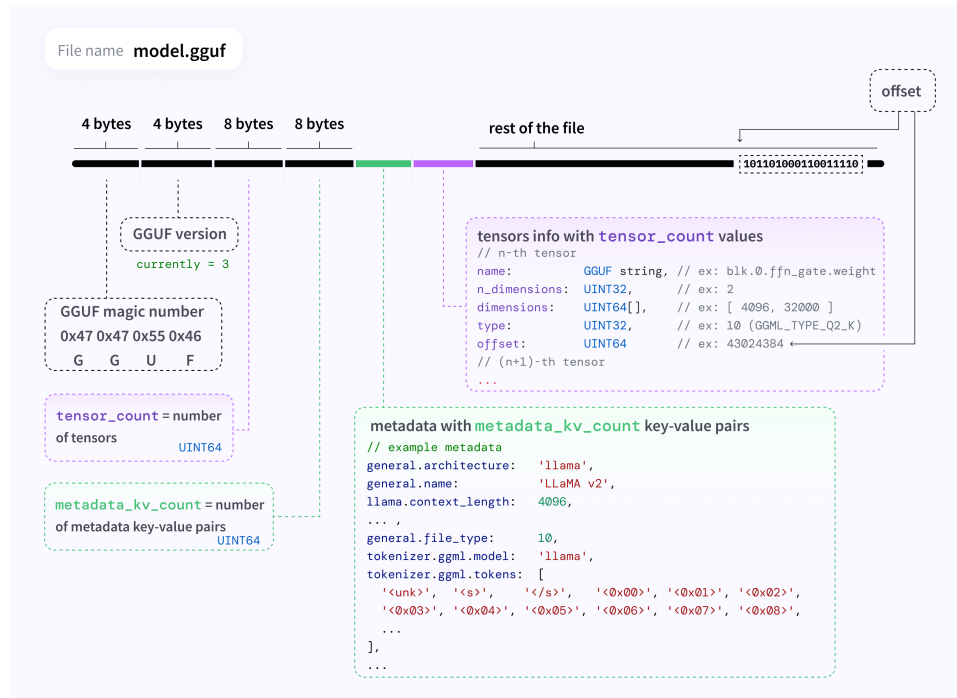


Figure 8: GGUF [1] binary file layout

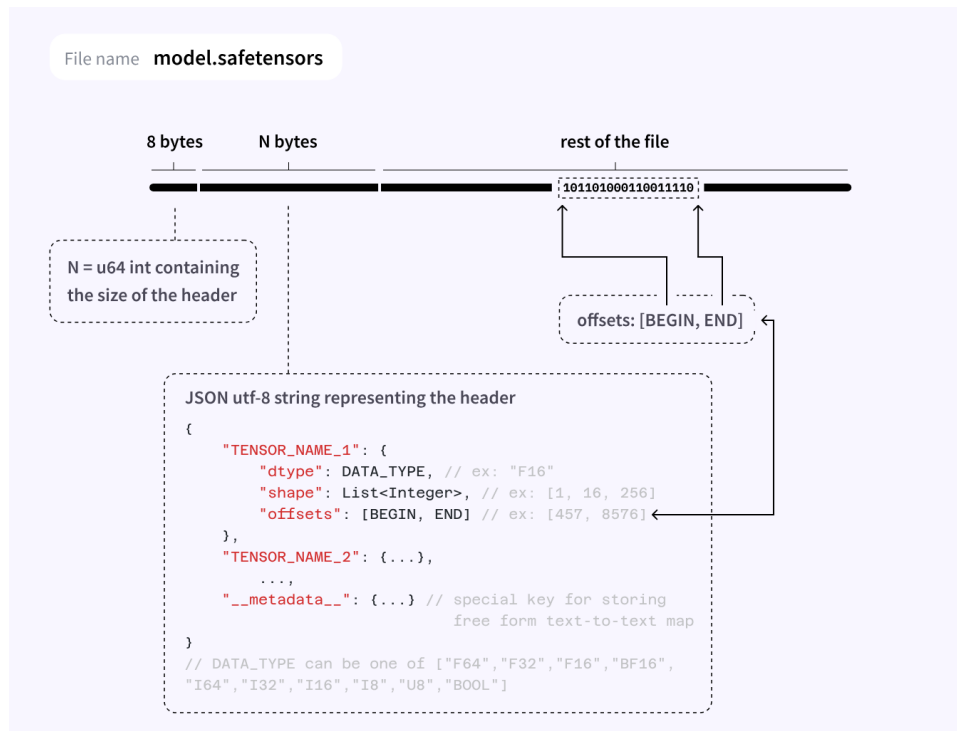


Figure 9: Safetensors architecture

7.2 From C++ to HLS

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 # 1) Vitis environment
5 source /tools/Xilinx/2025.1/Vitis/settings64.sh
6 export XILINX_PLATFORM_REPO_PATHS=/tools/Xilinx/2025.1/Vitis/
   base_platforms
7 PLAT=zcu106_dpu                                # platform name as it
   appears in "platform list"
8
9 # 2) Build directory
10 cd "$(dirname "$0")"
11 mkdir -p build_fpga
12 cd build_fpga
13
14 # 3) Compile the kernels (hardware) -----
15 for k in matmul mul rmsnorm softmax add rope; do
16     v++ -c -t hw --platform "$PLAT" \
17         -k kernel_${k} ../src/kernel_${k}.cpp \
18         -o kernel_${k}.xo
19 done
20
21 # 4) Link: generate the XCLBIN -----
22 v++ -l -t hw --platform "$PLAT" \
23     kernel_*.xo \
24     -o binary_container_1.xclbin
25
26 echo -e "\n Build finished: binary_container_1.xclbin is ready."
```


References

- [1] Georgi Gerganov. Gguf format specification, 2024. Accessed: 2025-07-14.
- [2] Turing Inc. Swan: A lightweight language model execution environment using fpga. <https://github.com/turingmotors/swan>, 2024. Accessed: 2025-07-22.
- [3] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.